משחק חניה

פרויקט מסכם — תכנות מונחה עצמים 2 (OOP2)

המכללה האקדמית הדסה

---

1. פרטי הסטודנטים

- **ידידיה אטלן** — ת"ז: 327929428
- **אלישע צרלין** — ת"ז: 325419075

2. הסבר כללי על הפרויקט

הפרויקט הוא משחק חניה דו-ממדי הכתוב ב-C++ מודרני עם ספריית SFML 3. השחקן נוהג ברכב הנע לפי מודל פיזיקלי, ומטרתו להחנות את הרכב במקום החניה המסומן — במיקום ובזווית הנכונים — מבלי להתנגש באף אחד מהמכשולים הפזורים בשלב (קירות, רכבים חונים, חרוטים, מחסומים, איי דשא ועוד).

המשחק עוטף את ליבת המשחקיות בחוויה מלאה: מספר מסכים (תפריט ראשי, בחירת שלב, מסך משחק, מסך עזרה ומסך קרדיטים), שישה שלבים הנטענים מקובצי JSON חיצוניים, מערכת ניקוד עם דירוג כוכבים, שמירת התקדמות בין הרצות (פתיחת שלבים ושמירת זמנים שיא), מערכת אנימציה לאפקטים ויזואליים (ענן עשן הנפלט בנקודת ההתנגשות), ואפקטי קול ומוזיקה.

מבחינה תכנונית, ההחלטה המנחה לאורך כל הפרויקט הייתה לבחור כל תבנית עיצוב וכל אלגוריתם **לפי הבעיה הספציפית** שהם פותרים — ולא להחיל תבנית "כי כך מקובל". סעיפים 5 ו-6 מוקדשים בדיוק לכך: הם מסבירים את ההחלטות הלא-טריוויאליות, מתי בחרנו בכיוון אחד ולמה דחינו את החלופה.

3. רשימת הקבצים שיצרנו

הפרויקט מאורגן בתיקיות לפי תחום אחריות. להלן הקבצים העיקריים והתפקיד של כל אחד.

ליבה (core)

- **main.cpp** — נקודת הכניסה; טוענת את המשאבים ומריצה את המשחק.
- **Game** — מנהלת את החלון, את מחסנית המסכים ואת הלולאה הראשית.
- **LevelManager** — מריצה שלב בודד: עדכון, ציור, ובדיקות ניצחון/הפסד.

אובייקטים (objects)

- **GameObject** — מחלקת בסיס מופשטת לכל אובייקט במשחק.
- **Car** — רכב השחקן: תנועה, היגוי, פיזיקת נסיעה וסיבוב גלגלים.
- **StaticObject** — אובייקט סטטי כללי (קיר, רכב חונה, חרוט וכו').

אנימציה ואפקטים (animation)

- **Animation** — מחלקת בסיס מופשטת: מנגנון התזמון הטהור של sprite sheet (איזה פריים מוצג, ומתי להתקדם לבא). אינה יודעת דבר על sprite או על ציור.
- **SmokeAnimation** — מימוש קונקרטי של אנימציית עשן (יורש מ-Animation); מחזיק את ה-sprite ומצייר אותו.
- **EffectManager** — מנהל אוסף האפקטים החיים: הוספה, עדכון, ציור, ומחיקת אפקטים שהסתיימו.
- **SmokeAniSettings** — קבועי הכיוונון של אנימציית העשן (הפריימים, הקצב, הגודל).

אסטרטגיות שליטה (strategies)

- **ControlStrategy** — ממשק מופשט לשליטה ברכב.
- **KeyboardStrategy** — מימוש קונקרטי של שליטה במקלדת.

נתונים, טעינה ומשאבים (data)

- **Factory** — רישום ויצירת אובייקטים לפי סוג.
- **Loader** — קריאת קובץ שלב JSON והמרתו ל-LevelDescriptor.
- **LevelBuilder** — בניית האובייקטים בפועל מתוך התיאור.
- **Playlist** — ניהול רשימת השלבים (levels.json).
- **Progress** — שמירה וטעינה של התקדמות השחקן.
- **SoundManager** — ניהול אפקטי קול ומוזיקה.
- **ResourceManager** — טעינה וקאשינג מרוכז של טקסטורות, פונטים וקולות.
- **AssetPaths** — ריכוז נתיבי קבצי המשאבים במקום אחד.

משחקיות (gameplay)

- **HUD** — תצוגת המידע על המסך בזמן משחק.
- **Score** — חישוב הניקוד ודירוג הכוכבים.
- **Timer** — מדידת הזמן בשלב.

מסכים (screens)

- **Screen** — מחלקת בסיס מופשטת למסך.

- **MenuScreen** — התפריט הראשי.
- **LevelSelectScreen** — מסך בחירת שלב.
- **GameScreen** — מסך המשחק עצמו.
- **HelpScreen** — מסך הוראות ועזרה.
- **CreditsScreen** — מסך קרדיטים.

רכיבי ממשק (ui)

- **Button**, **Background**, **ClockWidget**, **LevelTile**, **Modal**, **StarRating** — רכיבי ממשק לשימוש חוזר.
- **ViewUtils** — פונקציות עזר ל-sf::View ולמיפוי קואורדינטות.

עזר (utility)

- **Collision** — בדיקות התנגשות בשיטת SAT, וחישוב נקודת ההתנגשות לאפקט העשן.
- **ParkingRules** — בדיקה האם הרכב חונה כראוי במקום החניה.
- **EffectMaker** — namespace של פונקציות חופשיות היוצרות אפקט מוכן (למשל עשן) לפי הקשר.
- **SpriteUtils** — פונקציות עזר ל-sprite (התאמת גודל וכו').
- **FileUtils** — פונקציות עזר לקריאת וכתיבת קבצים.
- **ObjectUtils** — המרה בין שמות סוגי אובייקטים לערכי enum.
- **enums/** — הגדרות enum: Objects, Effects, Fonts, Sounds, Musics, LevelState, UITextures.
- **structs/** — מבני נתונים: LevelData, LevelDescriptor, ObjectDescriptor.

4. מבני נתונים והחלטות תכנון

שלושת מבני התיאור (Descriptor)

לאורך כל צינור הטעינה הקפדנו להפריד בין "מה כתוב בקובץ" לבין "האובייקט החי במשחק". ההפרדה הזו מתבטאת בשלושה מבנים:

- **ObjectDescriptor** — מתאר אובייקט בודד כפי שנקרא מ-JSON: סוג (enum), קואורדינטות (x,y) וזווית. זהו "התיאור הגולמי", עוד לפני שנוצר אובייקט כלשהו.
- **LevelDescriptor** — מאגד תיאור של שלב שלם: תיאור הרכב, תיאור מקום החניה, ווקטור של תיאורי שאר האובייקטים.

- **LevelData** — התוצר שלאחר הבנייה: האובייקטים הפולימורפיים עצמם, מחולקים לחוסמים (m_blocksObj) ולקישוטיים / לא-חוסמים (m_decalsObj), ומוכנים לעדכון ולציור.

> **למה מפרידים Descriptor מ-Object?** שלב הטעינה אינו צריך להכיר את פרטי הבנייה של כל מחלקה ומחלקה, ושלב הבנייה אינו צריך להכיר את פורמט ה-JSON. בזכות ההפרדה, Loader ו-Factory יכולים להשתנות ולהתפתח באופן עצמאי לחלוטין זה מזה.

> **למה מפרידים אובייקטים חוסמים מקישוטיים כבר במבנה הנתונים?** לא כל אובייקט בשלב הוא מכשול — מעבר חצייה או קו הפרדה הם רקע ויזואלי בלבד. החלוקה לשני ווקטורים נפרדים מאפשרת ללולאת ההתנגשויות לבדוק רק את האובייקטים החוסמים, בלי לבזבז בדיקות על קישוטים, ובלי תנאי if בתוך הלולאה החמה.

היררכיית GameObject

מחלקת הבסיס המופשטת **GameObject** מגדירה את החוזה המשותף: getCorners(), getGlobalBounds(), getAngle() ו-update(). **Car** ו-**StaticObject** יורשים ממנה, וכל המערכת מחזיקה אותם דרך unique_ptr<GameObject> — מה שמאפשר טיפול אחיד ופולימורפי בכל סוגי האובייקטים דרך מצביע אחד לבסיס.

> **למה getCorners() נמצא בבסיס ולא רק ברכב?** בדיקת ההתנגשות פועלת על ארבע הפינות הגלובליות של אובייקט, והיא חייבת לעבוד באופן אחיד מול כל אובייקט מסובב — רכב מול קיר, רכב מול חרוט, וכן הלאה. לכן "ארבע הפינות" הוא חלק מהחוזה (contract) של מחלקת הבסיס, ולא פרט מימוש פנימי של הרכב.

תבנית Strategy לשליטה ברכב

הרכב אינו קורא למקלדת באופן ישיר. במקום זאת הוא מחזיק מצביע ל-**ControlStrategy** — ממשק מופשט עם שאלות כמו wantsGas(), wantsLeft(), wantsBrake() — ובפועל מוזרק לו **KeyboardStrategy**.

> **למה מפרידים את השליטה למחלקה נפרדת?** כך לוגיקת הרכב מנותקת לחלוטין ממקור הקלט. הרכב "שואל" את האסטרטגיה מה השחקן רוצה, בלי לדעת אם התשובה מגיעה ממקלדת, מבקר משחק, מ-AI או מרשת. להחליף או להוסיף שיטת שליטה אפשר **בלי לגעת בקוד הפיזיקה של הרכב** — בדיוק העיקרון של Open/Closed.

מאגר ה-Factory

ה-Factory ממומש כ-namespace שמחזיק טבלת רישום פנימית: std::map<Objects, Creator>, כאשר Creator היא std::function<unique_ptr<GameObject>(const ObjectDescriptor&)>. זו אחת ההחלטות המרכזיות בפרויקט, ושקלנו בה במכוון בין שתי חלופות.

למה ה-Factory הוא namespace של פונקציות חופשיות, ולא singleton? שקלנו לממש את ה-Factory כ-singleton, אך הגענו למסקנה שזו הייתה בחירה שגויה כאן: ל-Factory **אין שום מצב** (state) שצריך לנהל — הוא אינו אלא טבלת רישום. singleton מצדיק את עצמו כשנדרש מופע יחיד בעל מצב ומחזור-חיים שיש לשלוט בהם; כאן הוא היה רק מוסיף getInstance(), בנאי פרטי וניהול חיים — סיבוכיות ללא שום תועלת.

במקום זאת בחרנו ב-namespace עם **מפה סטטית מקומית בתוך פונקציה** (getCreatorRegistry()). זה נותן בדיוק את אותה גישה גלובלית ואת אותו רישום-עצמי, אך בלי כל הטקס של singleton.

יתרון נוסף, עדין יותר: מפה סטטית מקומית בתוך פונקציה פותרת את **בעיית סדר האתחול הסטטי** (static initialization order fiasco). המפה נוצרת בקריאה הראשונה אליה, ולכן הרישומים הסטטיים המגיעים מקבצים שונים תמיד פועלים נכון, ללא תלות בסדר טעינת הקבצים.

למה דווקא ResourceManager כן ממומש כ-singleton? ההחלטה כאן הפוכה — **ובכוונה**. ל-ResourceManager יש מצב כבד: מאגרי הטקסטורות, הפונטים והקולות, שחייבים להיטען פעם אחת בלבד ולהיות משותפים לכל חלקי המשחק. זהו בדיוק התרחיש שבו singleton הוא הכלי הנכון. ההשוואה בין שתי ההחלטות ההפוכות היא העיקר: בחרנו את התבנית לפי היתצאות מצב משותף — ולא מתוך הרגל.

רישום עצמי (Self-Registration)

כל סוג אובייקט רושם את עצמו ב-Factory באמצעות אתחול של משתנה bool סטטי גלובלי, למשל: static bool registery = factory::registerCreator(...).

למה משתמשים בערך החזרה של פונקציה כדי לאתחל bool סטטי? זהו טריק שמנצל את העובדה שאתחול של משתנה סטטי גלובלי מריץ קוד עוד לפני main. כך כל מחלקת אובייקט "מצהירה על קיומה" בפני ה-Factory כבר בזמן טעינת התוכנית, ללא שום רשימת רישום מרכזית שצריך לתחזק. התוצאה המעשית: הוספת סוג אובייקט חדש פירושה קובץ חדש עם שורת רישום אחת — **בלי לגעת ב-Factory וב-LevelBuilder כלל**. זהו עקרון ה-Open/Closed הלכה למעשה.

שמירת מצב מתמשך — Progress

Progress (אף הוא singleton, ומאותה סיבה — מצב יחיד ומשותף) שומר את התקדמות השחקן אל קובץ progress.json שליד קובץ ההרצה: כמה שלבים נפתחו, ומהו הזמן הטוב ביותר בכל שלב. שלב 1 פתוח תמיד, וסיום שלב פותח את הבא אחריו.

מערכת האנימציה — שלוש שכבות אחריות

אנימציית sprite sheet היא במהותה דבר פשוט: תמונה אחת המכילה רצף פריימים, וכל רגע מציגים ממנה "חלון" אחד (textureRect) ומחליפים אותו לפי הזמן. אך כדי שהמערכת תהיה ניתנת להרחבה ולא "גוש" אחד, פירקנו אותה לשלוש שכבות, כשכל אחת אחראית על דבר **אחד**:

- **Animation** (בסיס) — **מנגנון התזמון בלבד**. מחזיק את ה-frames, את האינדקס הנוכחי ואת השעון הפנימי, ומחליט מתי להתקדם לפריים הבא. אינו יודע דבר על sprite, על ציור או על "מהו עשן".
- **SmokeAnimation** (יורש) — **הסוג הקונקרטי**. מוסיף את ה-sprite, מסנכרן אותו לפריים הנוכחי, ויודע לצייר את עצמו.
- **EffectManager** (מנהל) — **ניהול האוסף**. מחזיק את כל האפקטים החיים, מעדכן ומצייר את כולם, ומוחק את אלה שהסתיימו.

למה הבסיס Animation "טיפש" בכוונה — ואינו יודע דבר על sprite? הפרדנו את מנגנון התזמון (פריימים מול זמן) מן הציור (sprite ומיקום). מנגנון התזמון זהה לכל אנימציה אפשרית — עשן, פיצוץ, ניצוצות — בעוד שהציור הוא הדבר המשתנה. כשהבסיס אינו מכיר sprite, אותו מנגנון תזמון מדויק ישמש כל אנימציה עתידית, בלי שכפול ולו שורה אחת.

למה הבסיס קורא ליורש דרך onMoveFrame() (Template Method)? זוהי הנקודה העדינה ביותר במערכת. ה-update של הבסיס מנהל את כל הזרימה — צובר זמן, ומחליט מתי הגיע הרגע להחליף פריים. אך מה לעשות ברגע ההחלפה (לסנכרן את ה-sprite) שייך ליורש. לכן הבסיס, מיד לאחר שהחליף אינדקס, **קורא לפונקציה הווירטואלית** onMoveFrame(), וה-SmokeAnimation דורס אותה ומעדכן את ה-sprite שלו. כך הזרימה הכללית נכתבת פעם אחת בבסיס, וכל יורש "נתלה" רק על הרגע שמעניין אותו. זהו היפוך-שליטה (Inversion of Control): הבסיס שולט, היורש משלים.

למה המצב (loop מול חד-פעמי) הוא bool, אך הסוג (עשן) הוא ירושה? זיהינו כאן **שני צירים שונים** של שונות: ציר הסוג (עשן, פיצוץ — נבדלים בפריימים ובציור) וציר המצב (loop מול חד-פעמי — נבדלים רק בהתנהגות בסוף הרצף). את הסוג מימשנו **בירושה** (SmokeAnimation), ואת המצב ב**דגל bool** פשוט. לו מימשנו גם את המצב בירושה, היינו נדרשים למחלקה לכל שילוב (LoopSmoke, OnceSmoke, LoopExplosion...) — פיצוץ קומבינטורי של מחלקות. הכלל שהנחה אותנו: ציר אחד בירושה, ציר אחד כנתון.

מנהל האפקטים — בעלות, פולימורפיזם ומחיקה אוטומטית

ה-**EffectManager** מחזיק std::vector<std::unique_ptr<Animation>>. LevelManager מדבר רק איתו: "הוסף עשן כאן", "עדכן הכל", "צייר הכל" — וכל הניהול מוסתר מאחורי הממשק הקטן הזה.

למה מצביעים (unique_ptr) ולא אובייקטים בתוך ה-vector? vector<Animation> של אובייקטים "שלמים" היה **חותך** (slicing) כל SmokeAnimation שנדחף אליו — שומר רק את חלק הבסיס וזורק את ה-sprite ואת ההתנהגות. מצביע, לעומת זאת, שומר את האובייקט המלא, וכך קריאה פולימורפית (effect->update()) מריצה את הגרסה הנכונה של כל סוג. בחרנו דווקא unique_ptr (ולא מצביע גולמי) כי הוא **בעלים** של האובייקט: כשאפקט מוסר מה-vector, הוא נמחק מהזיכרון אוטומטית, ללא delete ידני ובלי דליפות.

למה ל-Animation יש destructor וירטואלי? האפקטים נמחקים דרך מצביע לבסיס (unique_ptr<Animation>). ללא destructor וירטואלי, delete על מצביע בסיס היה קורא רק ל-destructor של Animation — וה-sprite (ושאר משאבי היורש) היו **דולפים**. ה-virtual מבטיח שהמחיקה מגיעה ל-SmokeAnimation האמיתי ומנקה אותו במלואו.

למה האפקטים נוצרים ב-EffectMaker (namespace), ולא ב-Factory? לכאורה נכון היה לחקות את תבנית ה-Factory של האובייקטים גם כאן. אך ה-Factory מצדיק את עצמו רק כשהסוג נקבע **בזמן ריצה מנתון חיצוני** — שם, סוג האובייקט מגיע כמחרוזת מתוך ה-JSON של השלב, ואיננו ידוע מראש. האפקטים, לעומת זאת, נוצרים **מתוך הקוד** ברגע ההתנגשות: אנו יודעים שזה עשן. לכן רישום-עצמי וטבלת Creator-ים דינמית הם מנגנון שאיננו זקוקים לו. במקומם, namespace של פונקציות חופשיות (effectMaker::makeSmokeLoop) מרכז את ידע היצירה — וזהו עדיין Open/Closed: אפקט חדש = פונקציה חדשה ושורת include, בלי לגעת בקוד הקיים.

5. אלגוריתמים הראויים לציון

א. זיהוי התנגשויות — SAT (Separating Axis Theorem)

Collision::overlaps בודק חפיפה בין שני מלבנים מסובבים. לכל מלבן נלקחים שני צירים (צלע אחת והניצב לה), ועל כל אחד מארבעת הצירים מוקרנים (באמצעות dot product) שני המלבנים אל קטע [min, max]. אם קיים **ולו ציר אחד** שבו ההקרנות אינן נחתכות — מובטח שאין התנגשות, והבדיקה מסתיימת מיד.

למה בחרנו SAT ולא AABB? AABB (תיבה תוחמת מיושרת-צירים) הוא זול ופשוט, אך הוא בודק תיבה ישרה סביב הצורה. הרכב שלנו **מסתובב**, ותיבה ישרה סביב רכב מסובב "מנפחת" אותו — היא הייתה מדווחת על התנגשות גם כאשר הרכב רק קרוב אלכסונית לקיר, בלי מגע אמיתי. SAT, לעומת זאת, בודק חפיפה לאורך הצירים האמיתיים של הצורות המסובבות, ולכן הוא מדויק עבור רכב בכל זווית. שילמנו במעט חישוב נוסף (הקרנות על ארבעה צירים) וקיבלנו תמורתו נכונות — וזה קריטי במשחק שבו כל התנגשות גוררת הפסד מיידי.

למה מכווצים את תיבת ההתנגשות ב-HITBOX_SCALE? טקסטורות ה-PNG מכילות שוליים שקופים סביב הצורה הנראית. בלי טיפול, ה-SAT היה מתייחס גם לשוליים השקופים כחלק מהגוף, ומדווח על התנגשות "מהאוויר". לכן, לפני הבדיקה, אנו מכווצים את ארבע הפינות אל מרכז הצורה בפקטור קבוע — כך תיבת ההתנגשות תואמת את מה שנראה בעין, וה"מגע" מתרחש רק כשהצורות הנראות באמת נוגעות.

ב. מודל תנועת הרכב (Bicycle / Kinematic Model)

תנועת הרכב מבוססת על מודל פיזיקלי המדמה רכב בעל סרן קדמי וסרן אחורי. בכל פריים, Car::drive מחשב תחילה את מיקום זנב הרכב לפי הכיוון (heading) הנוכחי, מעדכן את ה-heading בהתאם לזווית הגלגלים, למהירות ולמרחק בין הסרנים, ואז מחשב מחדש את מרכז הרכב מתוך מיקום הזנב לפי הכיוון החדש.

למה טרחנו עם מודל סרנים במקום "סיבוב במקום"? סיבוב פשוט של הספרייט סביב מרכזו נותן תחושה לא-מציאותית של "טנק" המסתובב סביב צירו. מודל הסרנים גורם לרכב להסתובב סביב הסרן האחורי, בדיוק כמו מכונית אמיתית — הזנב "נגרר" בעקבות החזית. דווקא ההתנהגות הזו היא מה שהופך את תמרון החניה עצמו לאתגר משמעותי, ולא לפעולה טכנית פשוטה.

ג. בדיקת חניה תקינה (ParkingRules::isParked)

הרכב נחשב "חונה" אם מתקיימים שני תנאים: יישור הזווית שלו לזווית מקום החניה, וקרבת מרכז הרכב למרכז החניה. הזווית מושווית בעזרת wrapSigned(), והמרחק נבדק בריבועו (lengthSquared() => OFFSET^2).

למה משווים מרחק בריבוע ולא מחשבים sqrt? השוואת מרחק בריבוע שקולה מתמטית להשוואת המרחק עצמו, אך היא **חוסכת חישוב שורש ריבועי** בכל פריים. אופטימיזציה קטנה אך נכונה, כאשר הבדיקה רצה ברציפות לאורך כל המשחק.

למה עוטפים את הפרש הזווית ב-wrapSigned? ללא העיטוף, רכב בזווית $359°$ ומטרה בזווית $1°$ היו נראים כבעלי הפרש של $358°$ — כאילו אינם מיושרים כלל — בעוד שבפועל המרחק הזוויתי ביניהם הוא $2°$ בלבד. עיטוף ההפרש לטווח $[-180°, 180°]$ מודד את המרחק הזוויתי האמיתי, הקצר ביותר.

ד. ניקוד ודירוג כוכבים (Score)

בסיום שלב, Score::compute ממיר את הזמן שחלף ואת מספר ההתנגשויות ששרד השחקן לניקוד מספרי ולדירוג של 1–3 כוכבים. הנוסחה: ניקוד בסיס שממנו מופחתות נקודות לכל שנייה ולכל התנגשות, עם רצפה תחתונה. דירוג הכוכבים נקבע לפי ספי זמן.

למה כל הספים והקנסות מרוכזים ב-ScoreSettings.h? ערכי הכיוונון (ניקוד בסיס, קנס לשנייה, קנס להתנגשות, ספי הכוכבים) אינם "קבועי קסם" מפוזרים בקוד, אלא constexpr מרוכזים בקובץ הגדרות אחד. כך אפשר לאזן את קושי המשחק ואת תחושת התגמול בלי לגעת בלוגיקה עצמה.

ה. מיקום אפקט העשן — נקודת ההתנגשות (hitPointByClosestCorners)

ברגע ההתנגשות יש לקבוע היכן יופיע העשן. ה-SAT עונה על השאלה "האם יש חפיפה?" — אך **לא** על "היכן נגעו". לכן הוספנו פונקציה נפרדת המחשבת קירוב לנקודת המגע: היא עוברת על כל זוג פינות (פינה מן הרכב מול פינה מן המכשול), מוצאת את הזוג הקרוב ביותר, ומחזירה את הנקודה האמצעית ביניהן.

למה אמצע הפינות הקרובות — ולא אמצע המרכזים, ולא הרחבת ה-SAT? שקלנו שלוש דרכים. אמצע בין מרכזי האובייקטים פשוט, אך נשבר כשאחד האובייקטים גדול בהרבה או כשהמגע בפינה — הנקודה "נמשכת" הרחק מאזור המגע. מנגד, להרחיב את ה-SAT כך שיחזיר את עומק החדירה המדויק (MTV) היה משכתב מנגנון התנגשות **קריטי ובדוק**, ומסבך אותו — מחיר גבוה עבור אפקט ויזואלי שהעין ממילא לא תבחין בדיוקו. הבחירה ב-אמצע הפינות הקרובות היא נקודת האיזון: היא נשענת על אותן פינות שכבר חושבו, נותנת נקודה הקרובה מאוד למגע האמיתי, ואינה נוגעת כלל בלוגיקת ה-SAT. התאמנו את מאמץ הדיוק לרגישות התוצאה — קירוב טוב מספיק, בלי סיכון מיותר.

ו. השכבה הויזואלית רצה גם לאחר ההפסד

ברגע ההתנגשות מצב השלב הופך ל-Lost, וה-update הרגיל של השלב מפסיק (return מוקדם). אך אנו רוצים שהעשן ימשיך לפעום ברקע בזמן שמסך ההפסד מוצג. לכן עדכון האפקטים — m_effects.update(time) — מוצב **מעל** ה-return המוקדם, ב-LevelManager::update.

למה עדכון האפקטים מעל ה-return של מצב Lost? זוהי הבעה ישירה של עיקרון עמוק: **השכבה הויזואלית עצמאית מלוגיקת המשחק**. לוגיקת השלב קפאה (הרכב אינו זז, הזמן עצר), אך העולם הויזואלי ממשיך "לנשום". מכיוון שהפרדנו את האפקטים לשכבה נפרדת מההתחלה, התיקון הזה היה שורה אחת במקום הנכון — לולא ההפרדה, היינו נדרשים לפצל את מצב ה-Lost ולסבך את כל הזרימה. ארכיטקטורה טובה נמדדת לא כשהכול עובד, אלא ברגע שבו משהו משתנה.

ז. ניהול מסכים כמכונת-מצבים (Screen Stack)

Game מחזיק std::stack<unique_ptr<Screen>>. כל Screen מחזיר לאחר כל פריים ערך ScreenAction — אחד מתוך Push, Replace, Pop, Quit או None — וה-Game מפעיל את הפעולה המבוקשת על המחסנית.

למה מחסנית מסכים, ולא שרשרת if/else על "המסך הנוכחי"? המחסנית מאפשרת "חזרה אחורה" באופן טבעי לחלוטין: מסך העזרה נדחף (Push) מעל מסך המשחק, וכאשר סוגרים אותו (Pop) המשחק חוזר בדיוק למצב שבו היה, וממשיך משם (onResume). מעבר לכך, כל מסך מבודד

— הוא אינו יודע מי נמצא מעליו או מתחתיו, אלא רק מצהיר מה הוא רוצה שיקרה. הוספת מסך חדש אינה נוגעת בקוד של אף מסך אחר.

ח. מיפוי עולם↔מסך (Letterbox View)

המשחק עובד בעולם לוגי קבוע בגודל 1120×600, הממופה אל חלון המשחק באמצעות sf::View תוך שמירה על יחס הממדים (applyLetterboxView).

למה עולם לוגי קבוע במקום עבודה בפיקסלים של החלון? כל המיקומים — בקבצי השלבים, בבדיקות ההתנגשות ובציור — נשמרים בקואורדינטות עולם אחידות, ללא תלות בגודל החלון בפועל. כך אותו שלב נראה ומתנהג בדיוק אותו דבר בכל רזולוציה, והחלון יכול להשתנות בלי לשבור דבר.

6. באגים ידועים

לא ידועים באגים בעת ההגשה.

7. הערות אחרות

בנייה והרצה. הפרויקט בנוי עם CMake ותלוי ב-SFML 3 (מובאת אוטומטית דרך cmake/SFML.cmake). יש לבנות באמצעות ה-preset המוגדר בקובץ CMakePresets.json.

משאבים. קבצי המדיה (תמונות, קולות, פונטים) נמצאים תחת תיקיית resources ומועתקים אל תיקיית ה-build בזמן הבנייה. הנתיבים מרוכזים ב-AssetPaths.h. גיליון הפריימים של העשן נטען דרך ResourceManager תחת ה-enum Effects::SMOKE.

שלבים. קבצי השלבים הם JSON תחת resources/levels/, ורשימת השלבים הפעילים מוגדרת ב-levels.json. הוספת שלב חדש דורשת רק קובץ JSON נוסף ושורה ברשימה.

הרחבה. שני צירי ההרחבה המרכזיים בנויים על אותו עיקרון של Open/Closed: הוספת סוג אובייקט חדש דורשת מחלקה חדשה עם שורת רישום אחת ב-Factory; והוספת אפקט חדש דורשת מחלקת אנימציה חדשה (יורשת מ-Animation) ופונקציית יצירה ב-EffectMaker — ללא שינוי כלשהו בקוד הקיים בשני המקרים.

תודה על הבדיקה.